1.5 Case Study: Bài toán vẽ hình ASCII nghệ thuật (DrawFigures)

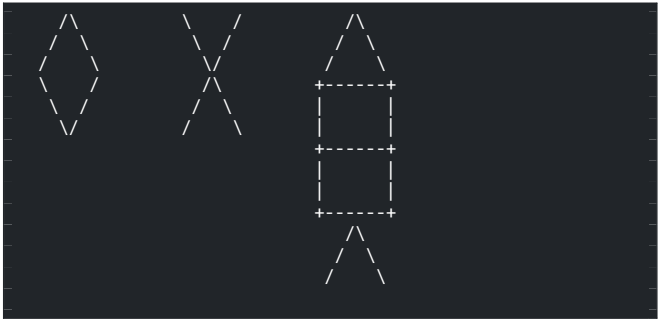
Yêu cầu bài toán (Output mong đợi)

Viết chương trình in ra 3 hình đồ họa ký tự ASCII:

1. **Hình Kim cương (Diamond)**
2. **Hình Chữ X (X)**
3. **Hình Tên lửa vũ trụ (Rocket)**

Các hình cách nhau một dòng trống.

Hình ảnh ASCII thực tế xuất ra Console:



Phiên bản 1 (DrawFigures1): Tiếp cận Ngây thơ (Naïve Approach)

Listing 4: DrawFigures1.java (Trích đoạn)

```java
public class DrawFigures1 {
    public static void main(String[] args) {
        // Ve kim cuong (Diamond)
        System.out.println("   /\\");
        System.out.println("  /  \\");
        System.out.println(" /    \\");
        System.out.println(" \\    /");
        System.out.println("  \\  /");
        System.out.println("   \\/");
        System.out.println();
        // Ve chu X
        System.out.println(" \\    /");
        System.out.println("  \\  /");
        System.out.println("   \\/");
        System.out.println("   /\\");
        // ... tiep tuc hang chuc dong lenh!
    }
}

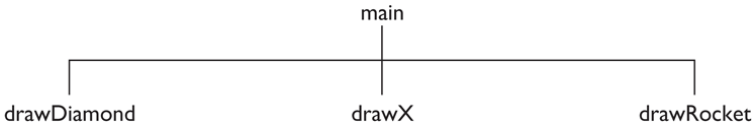
```

Đánh giá chất lượng thiết kế:

- **Số dòng lệnh:** Hơn 30 dòng lệnh `println` dồn hết vào hàm `main`.
- **Trùng lặp trầm trọng:** Cụm nón trên (*cone*) lặp lại 3 lần. Cụm nón dưới (*v*) lặp lại 2 lần. Cụm hình hộp lặp lại 2 lần.
- **Hậu quả bảo trì:** Nếu muốn mở rộng hình rộng hơn 1 dấu cách, ta phải đi sửa thủ công ở hàng chục vị trí khác nhau!

→ **Cần áp dụng Phân rã thủ tục cấp 1!**

Phiên bản 2 (DrawFigures2): Phân rã Cấp 1 theo Khối hình



Hình 1.3: Sơ đồ cấu trúc phân rã cấp 1 của DrawFigures2 (Trích giáo trình Chapter 1, Trang 92)

Cải tiến ở Phiên bản 2:

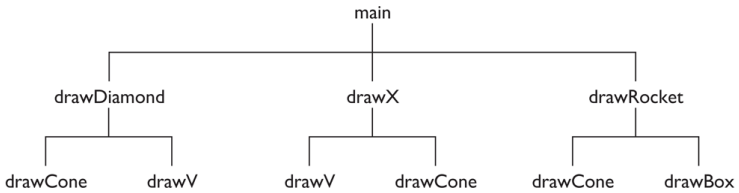
- Tách `main` thành 3 phương thức con:
  - `drawDiamond()`
  - `drawX()`
  - `drawRocket()`
- Hàm `main` bây giờ cực kỳ ngắn gọn và sáng sủa:

```
drawDiamond(); drawX(); drawRocket();
```

Hạn chế vẫn còn tồn tại:

- Bên trong thân của `drawDiamond`, `drawX`, và `drawRocket`, các câu lệnh in nón xuôi (`/\`) và nón ngược (`\/`) **vẫn bị lặp lại**!
- **Giải pháp triệt để:** Tiếp tục phân rã xuống cấp thứ 2 thành các hình khối cơ sở (*primitives*)!

Phiên bản 3 (DrawFigures3): Phân rã Hoàn chỉnh 2 Cấp (Triệt tiêu trùng lặp)



Hình 1.4: Sơ đồ cấu trúc phân rã hoàn chỉnh 2 cấp của DrawFigures3 (Trích giáo trình Chapter 1, Trang 96)

Cấu trúc 3 tầng chuẩn kiến trúc phần mềm:

- **Tầng 1 (Điều phối):** Phương thức `main()` nắm giữ bức tranh toàn cảnh cấp cao nhất.
- **Tầng 2 (Khối hình hoàn chỉnh):** `drawDiamond()`, `drawX()`, `drawRocket()`.
- **Tầng 3 (Các khối nguyên thủy tái sử dụng):** `drawCone()` (Nón trên), `drawV()` (Nón dưới), `drawBox()` (Thân hộp).

Mã nguồn hoàn chỉnh đạt chuẩn: `DrawFigures3.java`

Listing 5: Tầng 1 & Tầng 2

```java
public class DrawFigures3 {
    public static void main(String[] args) {
        drawDiamond();
        System.out.println();
        drawX();
        System.out.println();
        drawRocket();
    }
    public static void drawDiamond() {
        drawCone();
        drawV();
    }
    public static void drawX() {
        drawV();
        drawCone();
    }
    public static void drawRocket() {
        drawCone();
        drawBox();
        drawBox();
        drawCone();
    }
```

Listing 6: Tầng 3 (Primitives Reusable)

```java
    public static void drawCone() {
        System.out.println("   /\\");
        System.out.println("  /  \\");
        System.out.println(" /    \\");
    }
    public static void drawV() {
        System.out.println(" \\    /");
        System.out.println("  \\  /");
        System.out.println("   \\/");
    }
    public static void drawBox() {
        System.out.println("+------+");
        System.out.println("|      |");
        System.out.println("|      |");
        System.out.println("+------+");
    }
}

```

So sánh & Đánh giá 3 thế hệ thiết kế

Tiêu chí đánh giá	DrawFigures1 (Naïve)	DrawFigures2 (Cấp 1)	DrawFigures3 (Cấp 2)
Cấu trúc mã nguồn	Rối rắm, nguyên khối 1 hàm duy nhất.	Tách biệt từng hình lớn, dễ theo dõi hơn.	Phân tầng rõ rệt: Điều phối - Lắp ráp - Nguyên tử.
Mức độ trùng lặp	Rất cao (*Heavy redundancy*).	Vẫn lặp code vẽ nón và vẽ hộp.	**Triệt tiêu 100%** trùng lặp code in hình.
Khả năng tái sử dụng	0% (phải copy-paste dòng lệnh).	Chỉ tái sử dụng được cả hình kim cương hay chữ X.	**Tái sử dụng tối đa** mọi khối nguyên thủy.
Khả năng bảo trì	Sửa đổi 1 chi tiết phải sửa hàng chục chỗ.	Vẫn phải sửa nhiều nơi.	**Chỉ cần sửa 1 chỗ duy nhất** tại khối nguyên thủy!

[CHECKPOINT 3] Dự đoán luồng thực thi hàm lồng nhau

```java
public class TraceTest {
    public static void main(String[] args) {
        first();
        second();
    }
    public static void first() {
        System.out.println("1");
    }
    public static void second() {
        System.out.println("2");
        first();
        System.out.println("3");
    }
}

```

Thứ tự in trên màn hình là gì?

A. 1, 2, 3
B. 1, 2, 1, 3
C. 1, 2, 3, 1
D. 2, 1, 3

Giải thích:

1. `main` gọi `first()` → in **1**.
2. `main` gọi `second()`:
- In **2**.
- Gọi `first()` → in **1**.
- In **3**.
→ **Đáp án chính xác: B (1, 2, 1, 3)**

[HANDS-ON LAB 3] Mini-Project: Thiết kế chương trình Rocket/FightSong

Mini Project Tiết 3 (Thời gian: 20 phút – Đánh giá sản phẩm tại lớp)

Đề bài (Chọn 1 trong 2):

- **Lựa chọn A (Đồ họa ASCII):** Viết `SpaceRocket.java` vẽ một trạm không gian gồm 2 tên lửa đứng cạnh nhau, mỗi tên lửa gồm mũi nhọn, thân chứa biểu tượng `IT26M`, khoang động cơ, và ngọn lửa đáy. Yêu cầu **triệt tiêu 100% trùng lặp**.
- **Lựa chọn B (Bài hát cổ động - Exercise 12, p.125):** Viết chương trình `FightSong.java` in lời bài hát có các điệp khúc lặp lại bằng cách phân rã thành ít nhất 2 phương thức tĩnh phụ ngoài `main`.

Tiêu chí chấm điểm Lab:

1. Biên dịch và chạy hoàn hảo (`0 errors, 0 warnings`).
2. Cấu trúc phân rã rõ ràng (Tối thiểu 2 cấp phân rã).
3. Tuân thủ chuẩn đặt tên CamelCase và chú thích code.

Cấu trúc mẫu SpaceRocket:

- `drawNoseCone()`
- `drawBodySegment()`
- `drawThruster()`
- `drawFlame()`

Tổng kết toàn bộ kiến thức Chương 1 (Checklist Chuẩn đầu ra)

Mục tiêu CLO	Kiến thức cốt lõi đã làm chủ trong 3 tiết	Trạng thái
CLO 1.1	Hiểu phần cứng, nhị phân, vai trò của Bytecode và máy ảo JVM (WORA).	✓
CLO 1.2	Viết đúng cấu trúc class, hàm `main`, phân biệt `print`/`println`, dùng thành thạo ký tự thoát (`\n, \t, \", \\`).	✓
CLO 1.3	Nhận diện và phân biệt 3 loại lỗi: Cú pháp (Syntax), Thực thi (Runtime), Ngữ nghĩa (Logic); đọc hiểu thông báo lỗi compiler.	✓
CLO 1.4	Nắm vững tư duy phân rã Top-down; khai báo và gọi phương thức tĩnh; hiểu luồng điều khiển và ngăn xếp gọi hàm.	✓
CLO 1.5	Viết mã nguồn có cấu trúc module chuẩn, áp dụng nguyên lý DRY, biên dịch và chạy độc lập trên CLI và IDE.	✓

Bảng tra cứu nhanh cú pháp cốt lõi (Java Quick Cheatsheet)

Cấu trúc chương trình & Xuất màn hình

Khung chương trình:

```
public class TênLớp {
public static void main(String[] args) {
System.out.println("Nội dung");
}
}
```

Ký tự thoát:

`\n` (Xuống dòng) `\t` (Tab)
`\"` (Dấu ngoặc kép) `\\` (Gạch chéo ngược)

Phương thức tĩnh & Phân rã

Khai báo hàm:

```
public static void tênHàm() {
... các câu lệnh ...
}
```

Lời gọi hàm:

```
tênHàm();
```

Quy tắc vàng:

- Hàm không được lồng trong hàm khác.
- Đặt tên theo chuẩn `lowerCamelCase`.
- Triệt tiêu trùng lặp code (DRY).

Nhiệm vụ tự học & Bài tập về nhà cho lớp IT26M

1. Bài tập bắt buộc (Nộp trên hệ thống LMS)

- Đọc lại toàn bộ Chương 1 giáo trình (Trang 1 – 102).
- Làm các câu hỏi **Self-Check**:
  - Câu 6, 8, 10, 13 (Cú pháp & Escape Seq).
  - Câu 18, 19 (Tìm lỗi chương trình).
  - Câu 24, 26, 27 (Luồng điều khiển).
- Lập trình **Exercise 7 (Mantra)** và **Exercise 11 (TwoRockets)** trang 122–125.

2. Bài tập thử thách (Cộng điểm chuyên cần)

- **Programming Project 1 (MISSISSIPPI):** In chữ MISSISSIPPI dạng khối lớn (Block letters) với phân rã tối ưu (p.130).
- **Programming Project 3:** Bài hát dân gian tích lũy điệp khúc có cấu trúc phân tầng (p.131).

Chuẩn bị cho Buổi học kế tiếp:

Đọc trước **Chương 2: Primitive Data and Definite Loops** (Biến, Kiểu số nguyên/thực, Biểu thức toán học, và Vòng lặp `for`).

CẢM ƠN CÁC BẠN SINH VIÊN LỚP IT26M!

HỎI ĐÁP & THẢO LUẬN (Q&A)

Chúc các bạn có những giờ học lập trình Java say mê và hiệu quả!

Khoa Công nghệ Thông tin – Đại học UDA